

PySpark Theoretical Assignment 10

Spark SQL Query Processing, Catalyst Optimizer & Join Strategies

1. Explain the different stages of Spark SQL query processing

When we write a Spark SQL query or use the DataFrame API, Spark does not immediately execute it. Spark first creates and processes different plans to understand what the user wants, verify that it is valid, optimize it, and finally decide how to execute it on the cluster.

```
SQL / DataFrame Query
↓
Catalyst Parser
↓
Parsed Logical Plan
↓
Catalyst Analyzer
↓
Analyzed Logical Plan
↓
Catalyst Optimization Engine
↓
Optimized Logical Plan
↓
Physical Planning
↓
Physical Plan
↓
Execution
```

A. Parsed Logical Plan

The Parsed Logical Plan is the first representation created after Spark parses the query. At this stage, Spark understands the structure of the query, but it has not yet completely verified whether the tables, columns, functions, and data types actually exist.

```
employees.join(departments, 'dept_id').select('name', 'dept_name')
```

Conceptually, Spark sees operations such as Project → Join → employees / departments. The columns can still be unresolved at this stage.

Key idea: Parsed Logical Plan = What operations did the user request?

B. Analyzed Logical Plan

The Analyzer examines the parsed logical plan and resolves the objects used in the query. It checks whether tables and columns exist, determines which table a column belongs to, checks data types, resolves functions, and validates the query.

Example: `employees.join(departments, 'dept_id')` must resolve `employees.dept_id` and `departments.dept_id` and understand their actual data types.

Simple difference:

Parsed Logical Plan → What did the user ask?

Analyzed Logical Plan → Do I understand exactly what the user asked?

C. Optimized Logical Plan

After analysis, Spark tries to make the logical plan more efficient using the Catalyst optimization engine. Common optimizations include predicate pushdown, projection/column pruning, constant folding, removing unnecessary operations, simplifying expressions, and valid operation reordering.

```
df.filter(df.salary > 50000).select('name')
```

Spark can avoid carrying unnecessary columns and can logically apply the filter as early as possible. The goal is to produce the same result with less work.

D. Physical Plan

The Physical Plan describes how Spark will actually execute the optimized logical plan on the cluster. At this stage Spark selects physical operators such as Broadcast Hash Join, Sort-Merge Join, Shuffle Hash Join, and others depending on the query, join type, statistics, configuration, and hints.

Logical Plan: What should happen?

Physical Plan: How should Spark execute it?

```
joined_df = employees.join(departments, 'dept_id')
joined_df.explain(True)
```

The extended explanation can show Parsed Logical Plan, Analyzed Logical Plan, Optimized Logical Plan, and Physical Plan.

2. What is the Catalyst Optimizer in Spark SQL?

Catalyst is Spark SQL's query optimization framework. It takes a query and progressively transforms it into an efficient plan that Spark can execute.

SQL Query → Parser → Logical Plan → Analyzer → Analyzed Logical Plan → Optimizer → Optimized Logical Plan → Physical Planner → Physical Plan

A. Catalyst Parser

The parser takes a SQL query and converts it into a logical representation. It identifies operations such as SELECT, FROM, and WHERE and creates an unresolved logical plan.

```
SELECT name FROM employees WHERE salary > 50000;
```

In simple words: Parser = Converts the query into a structure Spark can understand.

B. Catalyst Analyzer

The Analyzer takes the parsed plan and resolves the objects in it. It determines the actual table, columns, data types, functions, and other references. If a referenced column does not exist, analysis can fail.

In simple words: Analyzer = Checks and resolves what the query is referring to.

C. Catalyst Optimization Engine

After the query is analyzed, Spark applies optimization rules to find a cheaper way to produce the same result. Examples include filtering data as early as possible and eliminating columns that are not needed.

In simple words: Optimizer = Finds a cheaper way to produce the same result.

Complete example

```
result = employees.filter(employees.salary > 50000).select('name')
```

DataFrame Query → Parsed / Unresolved Plan → Analyzer → Resolved Plan → Catalyst Optimizer → Optimized Logical Plan → Physical Planner → Physical Plan → Execution

3. Explain different join strategies used by Spark SQL

Spark can use different physical strategies to execute joins. The four strategies in this assignment are Sort-Merge Join, Broadcast Hash Join, Broadcast Nested Loop Join, and Shuffle Hash Join.

A. Sort-Merge Join

Sort-Merge Join is commonly used for joining large datasets. Spark shuffles both datasets according to the join key, sorts the relevant partitions, and then merges matching keys.

```
Employees → Shuffle by dept_id → Sort
Departments → Shuffle by dept_id → Sort
Then Spark merges matching keys.
```

Why is it called Sort-Merge? Because Spark sorts the data by the join key and then merges matching records.

Suitable for: Large datasets where broadcasting one side is not practical.

Simple idea: Both sides are shuffled and sorted, then matching keys are merged.

B. Broadcast Hash Join

Broadcast Hash Join is useful when one side of the join is small enough to broadcast. Spark sends the small dataset to the executors instead of performing a large join shuffle. The broadcast side is used to build a hash structure for matching.

```
Small Departments table → Broadcast to executors → Hash lookup against Employees
```

Advantage: It can avoid the large shuffle required by a Sort-Merge Join.

Simple idea: Send the small table to every executor and use it to join with the large table.

C. Broadcast Nested Loop Join

Broadcast Nested Loop Join broadcasts one side but does not rely on a normal hash-based equi-join. It can compare rows between the datasets and is useful for certain non-equality or special join conditions when one side is small enough to broadcast.

```
Example condition:  
employees.salary > departments.min_salary
```

A nested-loop style strategy can require many row comparisons, so it can become expensive when the broadcast side or the number of comparisons is large.

Simple idea: Broadcast one side and compare rows rather than relying on a normal hash-based equi-join.

D. Shuffle Hash Join

In a Shuffle Hash Join, Spark first shuffles both datasets according to the join key. After shuffling, Spark builds a hash table for the smaller side of each partition and uses the other side to find matching rows.

```
Employees → Shuffle by dept_id → Partitions  
Departments → Shuffle by dept_id → Partitions  
Then → Hash Join → Result
```

Difference from Sort-Merge Join:

Sort-Merge Join → Shuffle → Sort → Merge

Shuffle Hash Join → Shuffle → Build Hash Table → Hash Join

Suitable for: Equi-joins where the conditions and data characteristics make a hash-based strategy suitable.

Quick comparison of join strategies

Join Strategy	Main idea	Shuffle?	Typical situation
Sort-Merge Join	Shuffle + sort + merge	Yes	Large datasets
Broadcast Hash Join	Broadcast small side + hash lookup	No large join shuffle	One side is small
Broadcast Nested Loop Join	Broadcast one side + row comparisons	Usually avoids join shuffle	Non-equi/special joins with small side
Shuffle Hash Join	Shuffle + hash join	Yes	Suitable equi-joins

How does Spark decide which join to use?

Spark does not simply use one fixed join strategy. It considers the query, join type, join condition, data statistics, configuration, and join hints when provided.

Query → Join type → Join condition → Table/data statistics → Dataset sizes → Configuration → Join hints, if provided → Physical join strategy

If one side is small enough to broadcast, a Broadcast Hash Join may be selected. For large datasets where broadcasting is not practical, a Sort-Merge Join may be selected.

Complete picture

```
Spark SQL Query
↓
Parser
↓
Parsed Logical Plan
↓
Analyzer
↓
Analyzed Logical Plan
↓
Catalyst Optimizer
↓
Optimized Logical Plan
↓
Physical Planning
↓
Physical Plan
↓
Broadcast Hash / Sort-Merge / Shuffle Hash / Broadcast Nested Loop
↓
Execution
```

One-line memory trick

Parser → Understand the query

Analyzer → Resolve the query

Optimizer → Improve the query

Planner → Decide how to execute

Executor → Actually execute